

```
import zipfile
import os

# Path to the zip file
zip_file_path = "/content/dataset.zip"

# Directory to extract to
extract_to_path = "/content/"

# Ensure the extract directory exists
os.makedirs(extract_to_path, exist_ok=True)

# Open the zip file and extract its contents
with zipfile.ZipFile(zip_file_path, 'r') as zip_ref:
    zip_ref.extractall(extract_to_path)
    print(f"Extracted all files to {extract_to_path}")
```

 Extracted all files to /content/

```
import tensorflow as tf
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.layers import Dense, Flatten, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import ImageDataGenerator

import tensorflow as tf

# Set TensorFlow to only use GPU
gpus = tf.config.list_physical_devices('GPU')
if gpus:
    try:
        # Set memory growth to prevent TensorFlow from consuming all GPU memory
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
        print("GPU is configured for TensorFlow!")
    except RuntimeError as e:
        print(e)
else:
    print("No GPU available. Ensure proper GPU setup.")

# Load ResNet50 pre-trained on ImageNet
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Freeze the base model layers
base_model.trainable = False

# Add custom layers for banana quality classification
x = Flatten()(base_model.output)
x = Dense(128, activation='relu')(x)
x = Dropout(0.5)(x)
output = Dense(4, activation='softmax')(x) # Assuming 4 quality levels

# Define the model
model = Model(inputs=base_model.input, outputs=output)

# 2] MODEL TRAINING

# Compile the model
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

# Set up ImageDataGenerator for data augmentation and rescaling
train_datagen = ImageDataGenerator(rescale=1./255)
validation_datagen = ImageDataGenerator(rescale=1./255)

# Create the flow from directory using train and validation data
train_dataset = train_datagen.flow_from_directory(
    '/content/banana_classification/train',
    target_size=(224, 224), # Resizing images to match ResNet50 input
    batch_size=32,
    class_mode='sparse' # For sparse categorical labels
)

validation_dataset = validation_datagen.flow_from_directory(
    '/content/banana_classification/valid',
    target_size=(224, 224),
    batch_size=32,
```

```

    class_mode='sparse' # For sparse categorical labels
)

# Train the model
history = model.fit(
    train_dataset,
    validation_data=validation_dataset,
    epochs=10,
    batch_size=32
)

#Code for Model Evaluation

# Evaluate on the validation dataset
val_loss, val_accuracy = model.evaluate(validation_dataset)
print(f"Validation Loss: {val_loss}")
print(f"Validation Accuracy: {val_accuracy}")

#Evaluate Using a Separate Test Dataset

# Create a test dataset generator
test_datagen = ImageDataGenerator(rescale=1./255)

test_dataset = test_datagen.flow_from_directory(
    '/content/banana_classification/test', # Path to test dataset
    target_size=(224, 224),
    batch_size=32,
    class_mode='sparse'
)

# Evaluate on the test dataset
test_loss, test_accuracy = model.evaluate(test_dataset)
print(f"Test Loss: {test_loss}")
print(f"Test Accuracy: {test_accuracy}")

model.save('banana_features_new_model.h5')

```

GPU is configured for TensorFlow!
Found 11793 images belonging to 4 classes.
Found 1123 images belonging to 4 classes.
Epoch 1/10
/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:122: UserWarning: Your `PyDataset` c
self._warn_if_super_not_called()
369/369 ————— 63s 131ms/step - accuracy: 0.3162 - loss: 2.4027 - val_accuracy: 0.3455 - val_loss: 1.3499
Epoch 2/10
369/369 ————— 39s 105ms/step - accuracy: 0.3387 - loss: 1.3513 - val_accuracy: 0.3455 - val_loss: 1.3398
Epoch 3/10
369/369 ————— 37s 99ms/step - accuracy: 0.3434 - loss: 1.3477 - val_accuracy: 0.3455 - val_loss: 1.3377
Epoch 4/10
369/369 ————— 37s 100ms/step - accuracy: 0.3393 - loss: 1.3458 - val_accuracy: 0.3455 - val_loss: 1.3372
Epoch 5/10
369/369 ————— 37s 99ms/step - accuracy: 0.3481 - loss: 1.3418 - val_accuracy: 0.3455 - val_loss: 1.3371
Epoch 6/10
369/369 ————— 39s 105ms/step - accuracy: 0.3412 - loss: 1.3418 - val_accuracy: 0.3455 - val_loss: 1.3371
Epoch 7/10
369/369 ————— 39s 105ms/step - accuracy: 0.3403 - loss: 1.3433 - val_accuracy: 0.3455 - val_loss: 1.3370
Epoch 8/10
369/369 ————— 37s 99ms/step - accuracy: 0.3332 - loss: 1.3476 - val_accuracy: 0.3455 - val_loss: 1.3370
Epoch 9/10
369/369 ————— 43s 106ms/step - accuracy: 0.3383 - loss: 1.3466 - val_accuracy: 0.3455 - val_loss: 1.3369
Epoch 10/10
369/369 ————— 37s 99ms/step - accuracy: 0.3463 - loss: 1.3426 - val_accuracy: 0.3455 - val_loss: 1.3371
36/36 ————— 3s 84ms/step - accuracy: 0.3591 - loss: 1.3381
Validation Loss: 1.3370848894119263
Validation Accuracy: 0.3455031216144562
Found 562 images belonging to 4 classes.
18/18 ————— 6s 318ms/step - accuracy: 0.3019 - loss: 1.3879
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is c
Test Loss: 1.367017149925232
Test Accuracy: 0.3291814923286438

```

import tensorflow as tf
from tensorflow.keras.applications import ResNet50V2
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
import matplotlib.pyplot as plt

```

```

# GPU Configuration
gpus = tf.config.list_physical_devices('GPU')

```

```

if gpus:
    try:
        for gpu in gpus:
            tf.config.experimental.set_memory_growth(gpu, True)
            print("GPU is configured for TensorFlow!")
    except RuntimeError as e:
        print(e)
else:
    print("No GPU available. Ensure proper GPU setup.")

# Enhanced Data Augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.2,
    height_shift_range=0.2,
    horizontal_flip=True,
    zoom_range=0.15,
    shear_range=0.15,
    fill_mode='nearest'
)

validation_datagen = ImageDataGenerator(rescale=1./255)
test_datagen = ImageDataGenerator(rescale=1./255)

# Data Generators
train_dataset = train_datagen.flow_from_directory(
    '/content/banana_classification/train',
    target_size=(224, 224),
    batch_size=32,
    class_mode='sparse',
    shuffle=True
)

validation_dataset = validation_datagen.flow_from_directory(
    '/content/banana_classification/valid',
    target_size=(224, 224),
    batch_size=32,
    class_mode='sparse',
    shuffle=False
)

test_dataset = test_datagen.flow_from_directory(
    '/content/banana_classification/test',
    target_size=(224, 224),
    batch_size=32,
    class_mode='sparse',
    shuffle=False
)

# Model Architecture
base_model = ResNet50V2(weights='imagenet', include_top=False, input_shape=(224, 224, 3))

# Fine-tuning strategy
for layer in base_model.layers[:-30]: # Freeze all except last 30 layers
    layer.trainable = False

# Model structure
x = base_model.output
x = GlobalAveragePooling2D()(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.5)(x)
x = Dense(128, activation='relu')(x)
x = Dropout(0.3)(x)
output = Dense(4, activation='softmax')(x)

model = Model(inputs=base_model.input, outputs=output)

# Optimizer with initial learning rate
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)

# Compile model
model.compile(
    optimizer=optimizer,
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy', tf.keras.metrics.SparseTopKCategoryicalAccuracy(k=2, name='top_2_accuracy')]
)

# Callbacks
callbacks = [
    EarlyStopping(
        monitor='val_loss',
        patience=10,
        min_delta=0.001,
        verbose=1,
        mode='min'
    )
]

```

```

        patience=3,
        restore_best_weights=True,
        verbose=1
    ),
    ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.2,
        patience=3,
        min_lr=1e-6,
        verbose=1
    ),
    ModelCheckpoint(
        'best_banana_model.keras',
        monitor='val_accuracy',
        save_best_only=True,
        mode='max',
        verbose=1
    )
]

# Training
history = model.fit(
    train_dataset,
    validation_data=validation_dataset,
    epochs=30,
    callbacks=callbacks,
    verbose=1
)

# Plotting function
def plot_training_history(history):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

    # Plot accuracy
    ax1.plot(history.history['accuracy'], label='Training Accuracy')
    ax1.plot(history.history['val_accuracy'], label='Validation Accuracy')
    ax1.set_title('Model Accuracy')
    ax1.set_xlabel('Epoch')
    ax1.set_ylabel('Accuracy')
    ax1.legend()

    # Plot loss
    ax2.plot(history.history['loss'], label='Training Loss')
    ax2.plot(history.history['val_loss'], label='Validation Loss')
    ax2.set_title('Model Loss')
    ax2.set_xlabel('Epoch')
    ax2.set_ylabel('Loss')
    ax2.legend()

    plt.tight_layout()
    plt.show()

# Plot training history
plot_training_history(history)

# Final evaluation
print("\nEvaluating on test dataset...")
test_loss, test_accuracy, test_top2_accuracy = model.evaluate(test_dataset, verbose=1)
print(f"\nFinal Test Results:")
print(f"Loss: {test_loss:.4f}")
print(f"Accuracy: {test_accuracy:.4f}")
print(f"Top-2 Accuracy: {test_top2_accuracy:.4f}")

# Save final model
model.save('final_banana_classifier.h5')

# Print model summary
print("\nModel Architecture:")
model.summary()

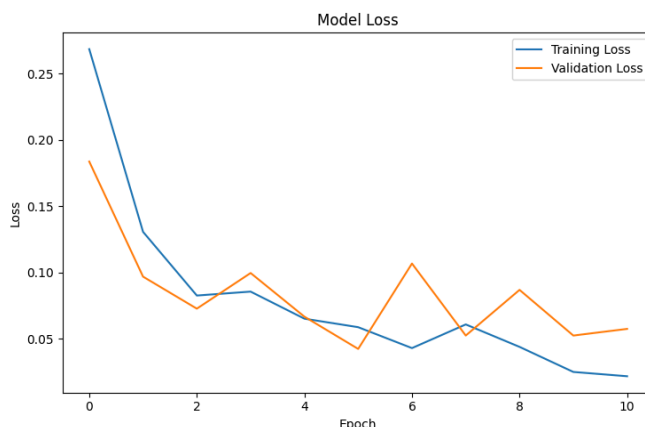
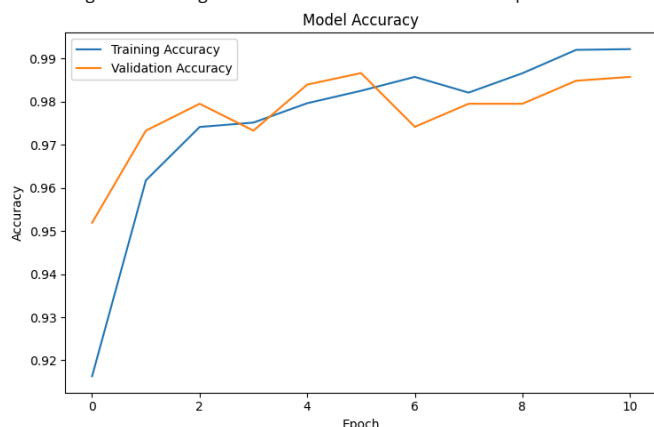
```

```

GPU is configured for TensorFlow!
Found 11793 images belonging to 4 classes.
Found 1123 images belonging to 4 classes.
Found 562 images belonging to 4 classes.
Epoch 1/30
/usr/local/lib/python3.11/dist-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:122: UserWarning: Your `PyDataset`
self._warn_if_super_not_called()
369/369 ━━━━━━━━━━━ 0s 416ms/step - accuracy: 0.8543 - loss: 0.4322 - top_2_accuracy: 0.9466
Epoch 1: val_accuracy improved from -inf to 0.95191, saving model to best_banana_model.keras
369/369 ━━━━━━━━━━━ 191s 457ms/step - accuracy: 0.8545 - loss: 0.4317 - top_2_accuracy: 0.9466 - val_accuracy: 0.9519 -
Epoch 2/30
369/369 ━━━━━━━━━━━ 0s 391ms/step - accuracy: 0.9571 - loss: 0.1475 - top_2_accuracy: 0.9957
Epoch 2: val_accuracy improved from 0.95191 to 0.97329, saving model to best_banana_model.keras
369/369 ━━━━━━━━━━━ 211s 504ms/step - accuracy: 0.9571 - loss: 0.1475 - top_2_accuracy: 0.9957 - val_accuracy: 0.9733 -
Epoch 3/30
369/369 ━━━━━━━━━━━ 0s 393ms/step - accuracy: 0.9744 - loss: 0.0794 - top_2_accuracy: 0.9980
Epoch 3: val_accuracy improved from 0.97329 to 0.97952, saving model to best_banana_model.keras
369/369 ━━━━━━━━━━━ 170s 417ms/step - accuracy: 0.9744 - loss: 0.0794 - top_2_accuracy: 0.9980 - val_accuracy: 0.9795 -
Epoch 4/30
369/369 ━━━━━━━━━━━ 0s 392ms/step - accuracy: 0.9730 - loss: 0.0918 - top_2_accuracy: 0.9967
Epoch 4: val_accuracy did not improve from 0.97952
369/369 ━━━━━━━━━━━ 149s 398ms/step - accuracy: 0.9730 - loss: 0.0918 - top_2_accuracy: 0.9967 - val_accuracy: 0.9733 -
Epoch 5/30
369/369 ━━━━━━━━━━━ 0s 395ms/step - accuracy: 0.9797 - loss: 0.0687 - top_2_accuracy: 0.9992
Epoch 5: val_accuracy improved from 0.97952 to 0.98397, saving model to best_banana_model.keras
369/369 ━━━━━━━━━━━ 185s 494ms/step - accuracy: 0.9797 - loss: 0.0687 - top_2_accuracy: 0.9992 - val_accuracy: 0.9840 -
Epoch 6/30
369/369 ━━━━━━━━━━━ 0s 390ms/step - accuracy: 0.9795 - loss: 0.0647 - top_2_accuracy: 0.9995
Epoch 6: val_accuracy improved from 0.98397 to 0.98664, saving model to best_banana_model.keras
369/369 ━━━━━━━━━━━ 156s 416ms/step - accuracy: 0.9795 - loss: 0.0646 - top_2_accuracy: 0.9995 - val_accuracy: 0.9866 -
Epoch 7/30
369/369 ━━━━━━━━━━━ 0s 390ms/step - accuracy: 0.9870 - loss: 0.0370 - top_2_accuracy: 0.9994
Epoch 7: val_accuracy did not improve from 0.98664
369/369 ━━━━━━━━━━━ 149s 396ms/step - accuracy: 0.9870 - loss: 0.0370 - top_2_accuracy: 0.9994 - val_accuracy: 0.9742 -
Epoch 8/30
369/369 ━━━━━━━━━━━ 0s 390ms/step - accuracy: 0.9814 - loss: 0.0641 - top_2_accuracy: 0.9985
Epoch 8: val_accuracy did not improve from 0.98664
369/369 ━━━━━━━━━━━ 150s 397ms/step - accuracy: 0.9814 - loss: 0.0641 - top_2_accuracy: 0.9985 - val_accuracy: 0.9795 -
Epoch 9/30
369/369 ━━━━━━━━━━━ 0s 392ms/step - accuracy: 0.9857 - loss: 0.0513 - top_2_accuracy: 0.9996
Epoch 9: ReduceLROnPlateau reducing learning rate to 0.00020000000949949026.

Epoch 9: val_accuracy did not improve from 0.98664
369/369 ━━━━━━━━━━━ 203s 398ms/step - accuracy: 0.9857 - loss: 0.0513 - top_2_accuracy: 0.9996 - val_accuracy: 0.9795 -
Epoch 10/30
369/369 ━━━━━━━━━━━ 0s 391ms/step - accuracy: 0.9927 - loss: 0.0242 - top_2_accuracy: 1.0000
Epoch 10: val_accuracy did not improve from 0.98664
369/369 ━━━━━━━━━━━ 149s 398ms/step - accuracy: 0.9927 - loss: 0.0242 - top_2_accuracy: 1.0000 - val_accuracy: 0.9849 -
Epoch 11/30
369/369 ━━━━━━━━━━━ 0s 394ms/step - accuracy: 0.9913 - loss: 0.0229 - top_2_accuracy: 0.9992
Epoch 11: val_accuracy did not improve from 0.98664
369/369 ━━━━━━━━━━━ 204s 402ms/step - accuracy: 0.9913 - loss: 0.0229 - top_2_accuracy: 0.9992 - val_accuracy: 0.9858 -
Epoch 11: early stopping
Restoring model weights from the end of the best epoch: 6.

```



Evaluating on test dataset...

```

18/18 ━━━━━━━━━━━ 5s 312ms/step - accuracy: 0.9890 - loss: 0.0445 - top_2_accuracy: 1.0000
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is

```

Final Test Results:
 Loss: 0.0526
 Accuracy: 0.9858
 Top-2 Accuracy: 1.0000

Model Architecture:
 Model: "functional_4"

Layer (type)	Output Shape	Param #	Connected to
input_layer_4 (InputLayer)	(None, 224, 224, 3)	0	-

conv1_pad (ZeroPadding2D)	(None, 230, 230, 3)	0	input_layer_4[0][0]
conv1_conv (Conv2D)	(None, 112, 112, 64)	9,472	conv1_pad[0][0]
pool1_pad (ZeroPadding2D)	(None, 114, 114, 64)	0	conv1_conv[0][0]
pool1_pool (MaxPooling2D)	(None, 56, 56, 64)	0	pool1_pad[0][0]
conv2_block1_preact_bn (BatchNormalization)	(None, 56, 56, 64)	256	pool1_pool[0][0]
conv2_block1_preact_relu (Activation)	(None, 56, 56, 64)	0	conv2_block1_preact_b...
conv2_block1_1_conv (Conv2D)	(None, 56, 56, 64)	4,096	conv2_block1_preact_r...
conv2_block1_1_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block1_1_conv[0...
conv2_block1_1_relu (Activation)	(None, 56, 56, 64)	0	conv2_block1_1_bn[0][...
conv2_block1_2_pad (ZeroPadding2D)	(None, 58, 58, 64)	0	conv2_block1_1_relu[0...
conv2_block1_2_conv (Conv2D)	(None, 56, 56, 64)	36,864	conv2_block1_2_pad[0]...
conv2_block1_2_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block1_2_conv[0...
conv2_block1_2_relu (Activation)	(None, 56, 56, 64)	0	conv2_block1_2_bn[0][...
conv2_block1_0_conv (Conv2D)	(None, 56, 56, 256)	16,640	conv2_block1_preact_r...
conv2_block1_3_conv (Conv2D)	(None, 56, 56, 256)	16,640	conv2_block1_2_relu[0...
conv2_block1_out (Add)	(None, 56, 56, 256)	0	conv2_block1_0_conv[0... conv2_block1_3_conv[0...
conv2_block2_preact_bn (BatchNormalization)	(None, 56, 56, 256)	1,024	conv2_block1_out[0][0]
conv2_block2_preact_relu (Activation)	(None, 56, 56, 256)	0	conv2_block2_preact_b...
conv2_block2_1_conv (Conv2D)	(None, 56, 56, 64)	16,384	conv2_block2_preact_r...
conv2_block2_1_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block2_1_conv[0...
conv2_block2_1_relu (Activation)	(None, 56, 56, 64)	0	conv2_block2_1_bn[0][...
conv2_block2_2_pad (ZeroPadding2D)	(None, 58, 58, 64)	0	conv2_block2_1_relu[0...
conv2_block2_2_conv (Conv2D)	(None, 56, 56, 64)	36,864	conv2_block2_2_pad[0]...
conv2_block2_2_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block2_2_conv[0...
conv2_block2_2_relu (Activation)	(None, 56, 56, 64)	0	conv2_block2_2_bn[0][...
conv2_block2_3_conv (Conv2D)	(None, 56, 56, 256)	16,640	conv2_block2_2_relu[0...
conv2_block2_out (Add)	(None, 56, 56, 256)	0	conv2_block1_out[0][0... conv2_block2_3_conv[0...
conv2_block3_preact_bn (BatchNormalization)	(None, 56, 56, 256)	1,024	conv2_block2_out[0][0]
conv2_block3_preact_relu (Activation)	(None, 56, 56, 256)	0	conv2_block3_preact_b...
conv2_block3_1_conv (Conv2D)	(None, 56, 56, 64)	16,384	conv2_block3_preact_r...
conv2_block3_1_bn (BatchNormalization)	(None, 56, 56, 64)	256	conv2_block3_1_conv[0...
conv2_block3_1_relu (Activation)	(None, 56, 56, 64)	0	conv2_block3_1_bn[0][...

conv2_block3_2_pad (ZeroPadding2D)	(None, 58, 58, 64)	0	conv2_block3_1_relu[0]...
conv2_block3_2_conv (Conv2D)	(None, 28, 28, 64)	36,864	conv2_block3_2_pad[0]...
conv2_block3_2_bn (BatchNormalization)	(None, 28, 28, 64)	256	conv2_block3_2_conv[0]...
conv2_block3_2_relu (Activation)	(None, 28, 28, 64)	0	conv2_block3_2_bn[0][...]...
max_pooling2d_9 (MaxPooling2D)	(None, 28, 28, 256)	0	conv2_block2_out[0][0]
conv2_block3_3_conv (Conv2D)	(None, 28, 28, 256)	16,640	conv2_block3_2_relu[0]...
conv2_block3_out (Add)	(None, 28, 28, 256)	0	max_pooling2d_9[0][0], conv2_block3_3_conv[0]...
conv3_block1_preact_bn (BatchNormalization)	(None, 28, 28, 256)	1,024	conv2_block3_out[0][0]
conv3_block1_preact_relu (Activation)	(None, 28, 28, 256)	0	conv3_block1_preact_b...
conv3_block1_1_conv (Conv2D)	(None, 28, 28, 128)	32,768	conv3_block1_preact_r...
conv3_block1_1_bn (BatchNormalization)	(None, 28, 28, 128)	512	conv3_block1_1_conv[0]...
conv3_block1_1_relu (Activation)	(None, 28, 28, 128)	0	conv3_block1_1_bn[0][...]...
conv3_block1_2_pad (ZeroPadding2D)	(None, 30, 30, 128)	0	conv3_block1_1_relu[0]...
conv3_block1_2_conv (Conv2D)	(None, 28, 28, 128)	147,456	conv3_block1_2_pad[0]...
conv3_block1_2_bn (BatchNormalization)	(None, 28, 28, 128)	512	conv3_block1_2_conv[0]...
conv3_block1_2_relu (Activation)	(None, 28, 28, 128)	0	conv3_block1_2_bn[0][...]...
conv3_block1_0_conv (Conv2D)	(None, 28, 28, 512)	131,584	conv3_block1_preact_r...
conv3_block1_3_conv (Conv2D)	(None, 28, 28, 512)	66,048	conv3_block1_2_relu[0]...
conv3_block1_out (Add)	(None, 28, 28, 512)	0	conv3_block1_0_conv[0]... conv3_block1_3_conv[0]...
conv3_block2_preact_bn (BatchNormalization)	(None, 28, 28, 512)	2,048	conv3_block1_out[0][0]
conv3_block2_preact_relu (Activation)	(None, 28, 28, 512)	0	conv3_block2_preact_b...
conv3_block2_1_conv (Conv2D)	(None, 28, 28, 128)	65,536	conv3_block2_preact_r...
conv3_block2_1_bn (BatchNormalization)	(None, 28, 28, 128)	512	conv3_block2_1_conv[0]...
conv3_block2_1_relu (Activation)	(None, 28, 28, 128)	0	conv3_block2_1_bn[0][...]...
conv3_block2_2_pad (ZeroPadding2D)	(None, 30, 30, 128)	0	conv3_block2_1_relu[0]...
conv3_block2_2_conv (Conv2D)	(None, 28, 28, 128)	147,456	conv3_block2_2_pad[0]...
conv3_block2_2_bn (BatchNormalization)	(None, 28, 28, 128)	512	conv3_block2_2_conv[0]...
conv3_block2_2_relu (Activation)	(None, 28, 28, 128)	0	conv3_block2_2_bn[0][...]...
conv3_block2_3_conv (Conv2D)	(None, 28, 28, 512)	66,048	conv3_block2_2_relu[0]...
conv3_block2_out (Add)	(None, 28, 28, 512)	0	conv3_block1_out[0][0]... conv3_block2_3_conv[0]...
conv3_block3_preact_bn (BatchNormalization)	(None, 28, 28, 512)	2,048	conv3_block2_out[0][0]

conv3_block3_preact_relu (Activation)	(None, 28, 28, 512)	0	conv3_block3_preact_b...
conv3_block3_1_conv (Conv2D)	(None, 28, 28, 128)	65,536	conv3_block3_preact_r...
conv3_block3_1_bn (BatchNormalization)	(None, 28, 28, 128)	512	conv3_block3_1_conv[0...
conv3_block3_1_relu (Activation)	(None, 28, 28, 128)	0	conv3_block3_1_bn[0][...
conv3_block3_2_pad (ZeroPadding2D)	(None, 30, 30, 128)	0	conv3_block3_1_relu[0...
conv3_block3_2_conv (Conv2D)	(None, 28, 28, 128)	147,456	conv3_block3_2_pad[0]...
conv3_block3_2_bn (BatchNormalization)	(None, 28, 28, 128)	512	conv3_block3_2_conv[0...
conv3_block3_2_relu (Activation)	(None, 28, 28, 128)	0	conv3_block3_2_bn[0][...
conv3_block3_3_conv (Conv2D)	(None, 28, 28, 512)	66,048	conv3_block3_2_relu[0...
conv3_block3_out (Add)	(None, 28, 28, 512)	0	conv3_block2_out[0][0... conv3_block3_3_conv[0...
conv3_block4_preact_bn (BatchNormalization)	(None, 28, 28, 512)	2,048	conv3_block3_out[0][0]
conv3_block4_preact_relu (Activation)	(None, 28, 28, 512)	0	conv3_block4_preact_b...
conv3_block4_1_conv (Conv2D)	(None, 28, 28, 128)	65,536	conv3_block4_preact_r...
conv3_block4_1_bn (BatchNormalization)	(None, 28, 28, 128)	512	conv3_block4_1_conv[0...
conv3_block4_1_relu (Activation)	(None, 28, 28, 128)	0	conv3_block4_1_bn[0][...
conv3_block4_2_pad (ZeroPadding2D)	(None, 30, 30, 128)	0	conv3_block4_1_relu[0...
conv3_block4_2_conv (Conv2D)	(None, 14, 14, 128)	147,456	conv3_block4_2_pad[0]...
conv3_block4_2_bn (BatchNormalization)	(None, 14, 14, 128)	512	conv3_block4_2_conv[0...
conv3_block4_2_relu (Activation)	(None, 14, 14, 128)	0	conv3_block4_2_bn[0][...
max_pooling2d_10 (MaxPooling2D)	(None, 14, 14, 512)	0	conv3_block3_out[0][0]
conv3_block4_3_conv (Conv2D)	(None, 14, 14, 512)	66,048	conv3_block4_2_relu[0...
conv3_block4_out (Add)	(None, 14, 14, 512)	0	max_pooling2d_10[0][0... conv3_block4_3_conv[0...
conv4_block1_preact_bn (BatchNormalization)	(None, 14, 14, 512)	2,048	conv3_block4_out[0][0]
conv4_block1_preact_relu (Activation)	(None, 14, 14, 512)	0	conv4_block1_preact_b...
conv4_block1_1_conv (Conv2D)	(None, 14, 14, 256)	131,072	conv4_block1_preact_r...
conv4_block1_1_bn (BatchNormalization)	(None, 14, 14, 256)	1,024	conv4_block1_1_conv[0...
conv4_block1_1_relu (Activation)	(None, 14, 14, 256)	0	conv4_block1_1_bn[0][...
conv4_block1_2_pad (ZeroPadding2D)	(None, 16, 16, 256)	0	conv4_block1_1_relu[0...
conv4_block1_2_conv (Conv2D)	(None, 14, 14, 256)	589,824	conv4_block1_2_pad[0]...
conv4_block1_2_bn (BatchNormalization)	(None, 14, 14, 256)	1,024	conv4_block1_2_conv[0...
conv4_block1_2_relu (Activation)	(None, 14, 14, 256)	0	conv4_block1_2_bn[0][...

conv4_block1_0_conv (Conv2D)	(None, 14, 14, 1024)	525,312	conv4_block1_preact_r...
conv4_block1_3_conv (Conv2D)	(None, 14, 14, 1024)	263,168	conv4_block1_2_relu[0...
conv4_block1_out (Add)	(None, 14, 14, 1024)	0	conv4_block1_0_conv[0... conv4_block1_3_conv[0...
conv4_block2_preact_bn (BatchNormalization)	(None, 14, 14, 1024)	4,096	conv4_block1_out[0][0]
conv4_block2_preact_relu (Activation)	(None, 14, 14, 1024)	0	conv4_block2_preact_b...
conv4_block2_1_conv (Conv2D)	(None, 14, 14, 256)	262,144	conv4_block2_preact_r...
conv4_block2_1_bn (BatchNormalization)	(None, 14, 14, 256)	1,024	conv4_block2_1_conv[0...
conv4_block2_1_relu (Activation)	(None, 14, 14, 256)	0	conv4_block2_1_bn[0][...
conv4_block2_2_pad (ZeroPadding2D)	(None, 16, 16, 256)	0	conv4_block2_1_relu[0...
conv4_block2_2_conv (Conv2D)	(None, 14, 14, 256)	589,824	conv4_block2_2_pad[0]...
conv4_block2_2_bn (BatchNormalization)	(None, 14, 14, 256)	1,024	conv4_block2_2_conv[0...
conv4_block2_2_relu (Activation)	(None, 14, 14, 256)	0	conv4_block2_2_bn[0][...
conv4_block2_3_conv (Conv2D)	(None, 14, 14, 1024)	263,168	conv4_block2_2_relu[0...
conv4_block2_out (Add)	(None, 14, 14, 1024)	0	conv4_block1_out[0][0... conv4_block2_3_conv[0...
conv4_block3_preact_bn (BatchNormalization)	(None, 14, 14, 1024)	4,096	conv4_block2_out[0][0]
conv4_block3_preact_relu (Activation)	(None, 14, 14, 1024)	0	conv4_block3_preact_b...
conv4_block3_1_conv (Conv2D)	(None, 14, 14, 256)	262,144	conv4_block3_preact_r...
conv4_block3_1_bn (BatchNormalization)	(None, 14, 14, 256)	1,024	conv4_block3_1_conv[0...
conv4_block3_1_relu (Activation)	(None, 14, 14, 256)	0	conv4_block3_1_bn[0][...
conv4_block3_2_pad (ZeroPadding2D)	(None, 16, 16, 256)	0	conv4_block3_1_relu[0...
conv4_block3_2_conv (Conv2D)	(None, 14, 14, 256)	589,824	conv4_block3_2_pad[0]...
conv4_block3_2_bn (BatchNormalization)	(None, 14, 14, 256)	1,024	conv4_block3_2_conv[0...
conv4_block3_2_relu (Activation)	(None, 14, 14, 256)	0	conv4_block3_2_bn[0][...
conv4_block3_3_conv (Conv2D)	(None, 14, 14, 1024)	263,168	conv4_block3_2_relu[0...
conv4_block3_out (Add)	(None, 14, 14, 1024)	0	conv4_block2_out[0][0... conv4_block3_3_conv[0...
conv4_block4_preact_bn (BatchNormalization)	(None, 14, 14, 1024)	4,096	conv4_block3_out[0][0]
conv4_block4_preact_relu (Activation)	(None, 14, 14, 1024)	0	conv4_block4_preact_b...
conv4_block4_1_conv (Conv2D)	(None, 14, 14, 256)	262,144	conv4_block4_preact_r...
conv4_block4_1_bn (BatchNormalization)	(None, 14, 14, 256)	1,024	conv4_block4_1_conv[0...
conv4_block4_1_relu (Activation)	(None, 14, 14, 256)	0	conv4_block4_1_bn[0][...
conv4_block4_2_pad	(None, 16, 16, 256)	0	conv4_block4_1_relu[0...

(ZeroPadding2D)			
conv4_block4_2_conv (Conv2D)	(None, 14, 14, 256)	589,824	conv4_block4_2_pad[0]...
conv4_block4_2_bn (BatchNormalization)	(None, 14, 14, 256)	1,024	conv4_block4_2_conv[0]...
conv4_block4_2_relu (Activation)	(None, 14, 14, 256)	0	conv4_block4_2_bn[0]...
conv4_block4_3_conv (Conv2D)	(None, 14, 14, 1024)	263,168	conv4_block4_2_relu[0]...
conv4_block4_out (Add)	(None, 14, 14, 1024)	0	conv4_block3_out[0][0]... conv4_block4_3_conv[0]...
conv4_block5_preact_bn (BatchNormalization)	(None, 14, 14, 1024)	4,096	conv4_block4_out[0][0]
conv4_block5_preact_relu (Activation)	(None, 14, 14, 1024)	0	conv4_block5_preact_b...
conv4_block5_1_conv (Conv2D)	(None, 14, 14, 256)	262,144	conv4_block5_preact_r...
conv4_block5_1_bn (BatchNormalization)	(None, 14, 14, 256)	1,024	conv4_block5_1_conv[0]...
conv4_block5_1_relu (Activation)	(None, 14, 14, 256)	0	conv4_block5_1_bn[0]...
conv4_block5_2_pad (ZeroPadding2D)	(None, 16, 16, 256)	0	conv4_block5_1_relu[0]...
conv4_block5_2_conv (Conv2D)	(None, 14, 14, 256)	589,824	conv4_block5_2_pad[0]...
conv4_block5_2_bn (BatchNormalization)	(None, 14, 14, 256)	1,024	conv4_block5_2_conv[0]...
conv4_block5_2_relu (Activation)	(None, 14, 14, 256)	0	conv4_block5_2_bn[0]...
conv4_block5_3_conv (Conv2D)	(None, 14, 14, 1024)	263,168	conv4_block5_2_relu[0]...
conv4_block5_out (Add)	(None, 14, 14, 1024)	0	conv4_block4_out[0][0]... conv4_block5_3_conv[0]...
conv4_block6_preact_bn (BatchNormalization)	(None, 14, 14, 1024)	4,096	conv4_block5_out[0][0]
conv4_block6_preact_relu (Activation)	(None, 14, 14, 1024)	0	conv4_block6_preact_b...
conv4_block6_1_conv (Conv2D)	(None, 14, 14, 256)	262,144	conv4_block6_preact_r...
conv4_block6_1_bn (BatchNormalization)	(None, 14, 14, 256)	1,024	conv4_block6_1_conv[0]...
conv4_block6_1_relu (Activation)	(None, 14, 14, 256)	0	conv4_block6_1_bn[0]...
conv4_block6_2_pad (ZeroPadding2D)	(None, 16, 16, 256)	0	conv4_block6_1_relu[0]...
conv4_block6_2_conv (Conv2D)	(None, 7, 7, 256)	589,824	conv4_block6_2_pad[0]...
conv4_block6_2_bn (BatchNormalization)	(None, 7, 7, 256)	1,024	conv4_block6_2_conv[0]...
conv4_block6_2_relu (Activation)	(None, 7, 7, 256)	0	conv4_block6_2_bn[0]...
max_pooling2d_11 (MaxPooling2D)	(None, 7, 7, 1024)	0	conv4_block5_out[0][0]
conv4_block6_3_conv (Conv2D)	(None, 7, 7, 1024)	263,168	conv4_block6_2_relu[0]...
conv4_block6_out (Add)	(None, 7, 7, 1024)	0	max_pooling2d_11[0][0]... conv4_block6_3_conv[0]...
conv5_block1_preact_bn (BatchNormalization)	(None, 7, 7, 1024)	4,096	conv4_block6_out[0][0]
conv5_block1_preact_relu (Activation)	(None, 7, 7, 1024)	0	conv5_block1_preact_b...
conv5_block1_1_conv (Conv2D)	(None, 7, 7, 512)	534,388	conv5_block1_preact_b...

conv5_block1_1_conv (Conv2D)	(None, 7, 7, 512)	2,048	conv5_block1_preact_r...
conv5_block1_1_bn (BatchNormalization)	(None, 7, 7, 512)	2,048	conv5_block1_1_conv[0]...
conv5_block1_1_relu (Activation)	(None, 7, 7, 512)	0	conv5_block1_1_bn[0][...]...
conv5_block1_2_pad (ZeroPadding2D)	(None, 9, 9, 512)	0	conv5_block1_1_relu[0]...
conv5_block1_2_conv (Conv2D)	(None, 7, 7, 512)	2,359,296	conv5_block1_2_pad[0]...
conv5_block1_2_bn (BatchNormalization)	(None, 7, 7, 512)	2,048	conv5_block1_2_conv[0]...
conv5_block1_2_relu (Activation)	(None, 7, 7, 512)	0	conv5_block1_2_bn[0][...]...
conv5_block1_0_conv (Conv2D)	(None, 7, 7, 2048)	2,099,200	conv5_block1_preact_r...
conv5_block1_3_conv (Conv2D)	(None, 7, 7, 2048)	1,050,624	conv5_block1_2_relu[0]...
conv5_block1_out (Add)	(None, 7, 7, 2048)	0	conv5_block1_0_conv[0]... conv5_block1_3_conv[0]...
conv5_block2_preact_bn (BatchNormalization)	(None, 7, 7, 2048)	8,192	conv5_block1_out[0][0]
conv5_block2_preact_relu (Activation)	(None, 7, 7, 2048)	0	conv5_block2_preact_b...
conv5_block2_1_conv (Conv2D)	(None, 7, 7, 512)	1,048,576	conv5_block2_preact_r...
conv5_block2_1_bn (BatchNormalization)	(None, 7, 7, 512)	2,048	conv5_block2_1_conv[0]...
conv5_block2_1_relu (Activation)	(None, 7, 7, 512)	0	conv5_block2_1_bn[0][...]...
conv5_block2_2_pad (ZeroPadding2D)	(None, 9, 9, 512)	0	conv5_block2_1_relu[0]...
conv5_block2_2_conv (Conv2D)	(None, 7, 7, 512)	2,359,296	conv5_block2_2_pad[0]...
conv5_block2_2_bn (BatchNormalization)	(None, 7, 7, 512)	2,048	conv5_block2_2_conv[0]...
conv5_block2_2_relu (Activation)	(None, 7, 7, 512)	0	conv5_block2_2_bn[0][...]...
conv5_block2_3_conv (Conv2D)	(None, 7, 7, 2048)	1,050,624	conv5_block2_2_relu[0]...
conv5_block2_out (Add)	(None, 7, 7, 2048)	0	conv5_block1_out[0][0]... conv5_block2_3_conv[0]...
conv5_block3_preact_bn (BatchNormalization)	(None, 7, 7, 2048)	8,192	conv5_block2_out[0][0]
conv5_block3_preact_relu (Activation)	(None, 7, 7, 2048)	0	conv5_block3_preact_b...
conv5_block3_1_conv (Conv2D)	(None, 7, 7, 512)	1,048,576	conv5_block3_preact_r...
conv5_block3_1_bn (BatchNormalization)	(None, 7, 7, 512)	2,048	conv5_block3_1_conv[0]...
conv5_block3_1_relu (Activation)	(None, 7, 7, 512)	0	conv5_block3_1_bn[0][...]...
conv5_block3_2_pad (ZeroPadding2D)	(None, 9, 9, 512)	0	conv5_block3_1_relu[0]...
conv5_block3_2_conv (Conv2D)	(None, 7, 7, 512)	2,359,296	conv5_block3_2_pad[0]...
conv5_block3_2_bn (BatchNormalization)	(None, 7, 7, 512)	2,048	conv5_block3_2_conv[0]...

```

# 1. Load and preprocess a single unseen image
import numpy as np
from tensorflow.keras.preprocessing import image

def predict_banana_quality(model, image_path):
    # Load and preprocess the image
    img = image.load_img(image_path, target_size=(224, 224))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array = img_array / 255.0 # Rescale like we did in training

    # Get predictions
    predictions = model.predict(img_array)

    # Get class names (assuming they're in alphabetical order from the directory)
    class_names = list(train_dataset.class_indices.keys())

    # Get top predictions
    top_indices = np.argsort(predictions[0])[::-1][:3] # Get top 3 predictions

    print("Predictions:")
    for idx in top_indices:
        print(f"{class_names[idx]}: {predictions[0][idx]*100:.2f}%")

    return predictions

# 2. Create a confusion matrix
from sklearn.metrics import confusion_matrix, classification_report
import seaborn as sns

def evaluate_model_performance():
    # Get true labels and predictions

    batch_size = test_dataset.batch_size
    test_steps = test_dataset.samples // batch_size

    true_labels = []
    predictions = []

    for i in range(len(test_dataset)):
        images, labels = test_dataset[i]
        preds = model.predict(images)
        pred_labels = np.argmax(preds, axis=1)

        true_labels.extend(labels)
        predictions.extend(pred_labels)

        if i >= test_steps-1:
            break

    # Create confusion matrix
    cm = confusion_matrix(true_labels, predictions)
    class_names = list(train_dataset.class_indices.keys())

    # Plot confusion matrix
    plt.figure(figsize=(10, 8))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names,
                yticklabels=class_names)
    plt.title('Confusion Matrix')
    plt.xlabel('Predicted')
    plt.ylabel('True')
    plt.show()

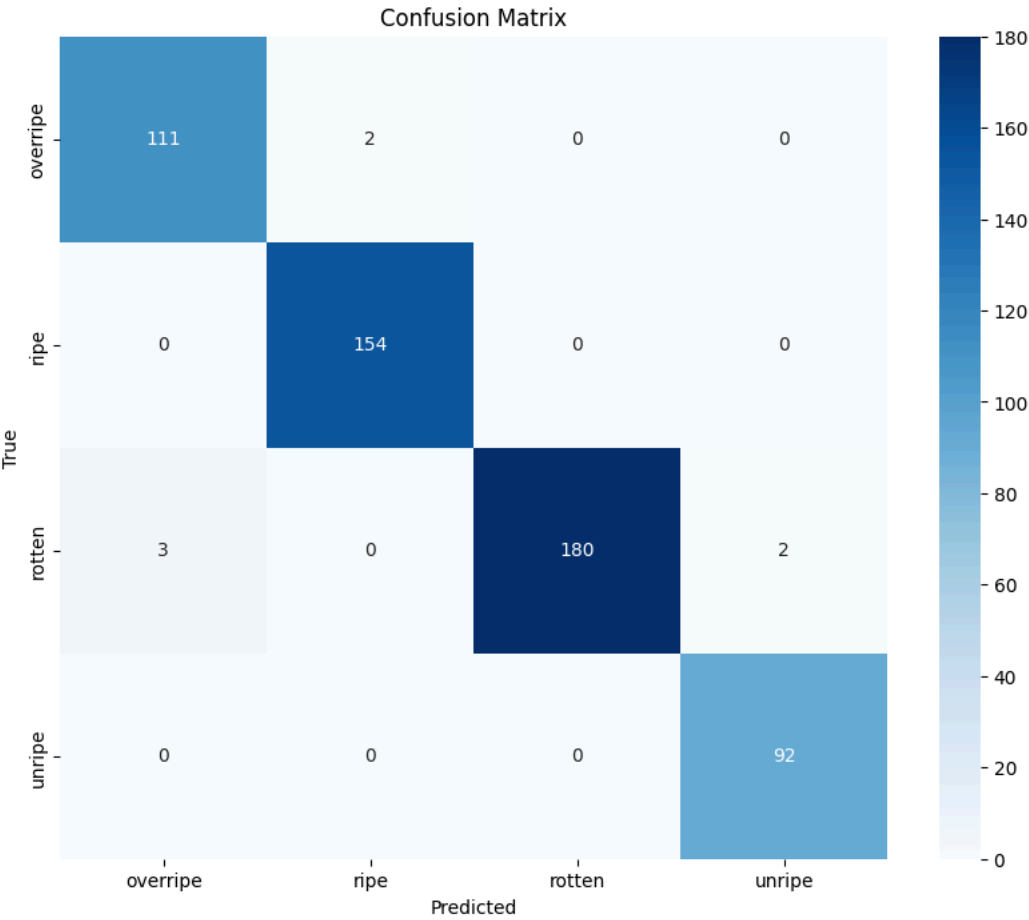
    # Print classification report
    print("\nClassification Report:")
    print(classification_report(true_labels, predictions,
                               target_names=class_names))

# Run the evaluation
evaluate_model_performance()

# Test with a new image
prediction = predict_banana_quality(model, '/content/402418916.jpg')

```

```
1/1 0s 31ms/step
1/1 0s 30ms/step
1/1 0s 40ms/step
1/1 0s 37ms/step
1/1 0s 42ms/step
1/1 0s 39ms/step
1/1 0s 39ms/step
1/1 0s 41ms/step
1/1 0s 31ms/step
1/1 0s 38ms/step
1/1 0s 38ms/step
1/1 0s 38ms/step
1/1 0s 38ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 30ms/step
1/1 0s 38ms/step
1/1 0s 29ms/step
```



Classification Report:

	precision	recall	f1-score	support
overripe	0.97	0.98	0.98	113
ripe	0.99	1.00	0.99	154
rotten	1.00	0.97	0.99	185
unripe	0.98	1.00	0.99	92
accuracy			0.99	544
macro avg	0.98	0.99	0.99	544

Start coding or [generate](#) with AI.

```
1/1 5s 5s/step
Predictions:
rotten: 100.00%
ripe: 0.00%
```